SMARTTASK: CONTEXT-AWARE MOBILE TO-DO LIST FOR INTELLIGENT TASK MANAGEMENT

Deployment Report

De Silva S.J.W. – IT21228612

25-26J-531

BSc (Hons) in Information Technology Specialising in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

TABLE OF CONTENTS

1 Executive Summary 1

1.1 Overview 1

1.2 System Diagram 1

1.3 Key Metrics 2

2 Deployment Objectives 4

2.1 Goals 4

2.2 Scope 4

3 Pre Deployment Activities 6

3.1 Environment Preparation 6

3.2 Testing Before Deployment 6

3.3 Backup Plan 7

4 Deployment Architecture and Runtime Environment 8

4.1 Application Deployment Type 8

4.2 Main Runtime Components 8

4.3 Local Data Storage and Privacy 9

5 Deployment Details 11

5.1 Date and Time 11

5.2 Team Members 11

5.3 Steps and Procedures 11

5.4 Tools and Technologies 12

6 Deployment Checklist 14

6.1 Pre Deployment Checklist 14

6.2 Deployment Checklist 14

6.3 Post Deployment Checklist 15

7 Issues and Resolutions 16
7.1 Encountered Issues 16
7.2 Resolutions 17
7.3 Impact Analysis 17
8 Performance and Monitoring 19
8.1 Performance Metrics 19
8.2 Monitoring Results 20
8.3 User Feedback 21
9 Team Details 23
Appendix 24

1 EXECUTIVE SUMMARY

1.1 Overview

This deployment report presents the final deployment of the SmartTask mobile application. SmartTask was developed as a context aware mobile task management system that supports users by identifying relevant tasks based on their real time situation. The application uses context data such as location, time, user activity, device state, calendar state, connectivity, and task properties to support task matching, task prioritization, and notification delivery.

The completed system was deployed as a native Android application developed using Java and Android Studio. The Minimum Viable Product (MVP) includes task creation, task editing, task deletion, task completion, voice based task input, map based location selection, context acquisition, context matching, dynamic task prioritization, adaptive notifications, and notification actions such as Done, Snooze, and Postpone. The application stores its main data locally using Room over SQLite, which supports offline use and improves privacy because task and context data do not need to be sent to an external server.

The deployment focused on proving the core SmartTask concept as a working MVP. Advanced behavioral learning and the full Data Logging and Insights dashboard were not completed as final user facing features. However, task state changes and notification actions were implemented, and these actions affected later task handling through updated task states. The system therefore demonstrates the main deployment goal of combining context awareness, dynamic prioritization, and adaptive notification delivery in a practical Android application.

1.2 System Diagram

The SmartTask deployment follows a modular mobile architecture. The main components are the Task Input module, Context Acquisition Engine, Context Matching module, Task Prioritization module, and Adaptive Notification system. The Data Logging and Insights component was included in the original design, but it was not implemented as a full user facing analytics module in the final MVP.

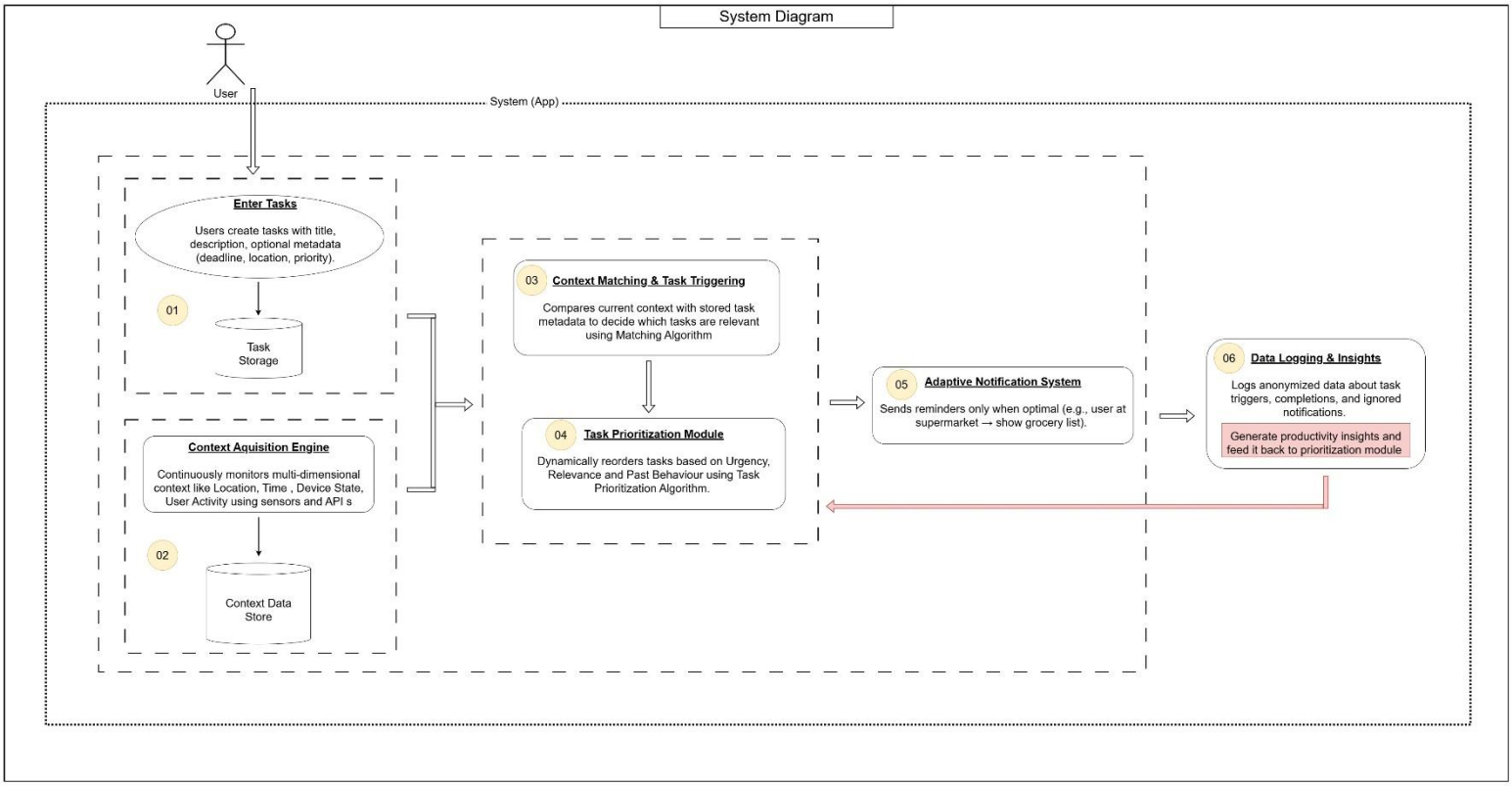


Figure 1: High level system architecture of SmartTask

The architecture shows how user tasks and context data move through the system. Task details are stored locally, context is collected from device services, context matching identifies relevant tasks, prioritization updates task order, and notification delivery sends reminders only when the task is suitable for the current situation. The complete source code of the SmartTask application is available through the GitHub repository provided in Appendix B.

1.3 Key Metrics

Metric	Value or Result	Description
Deployment success rate	100 percent	The final APK installed and launched successfully on the selected Android test device environment.
Application type	Native Android MVP	SmartTask was deployed as a Java based Android application.

Data storage model	Local Room database	Tasks, context snapshots, matching records, and related logs are stored locally.
Offline support	Supported for core functions	Task creation, editing, viewing, completion, and local prioritization operate without continuous internet connectivity.
Total participants in evaluation	8	The user evaluation was conducted with 4 students and 4 working professionals.
Total active testing days	109	Participants used the application for 12 to 15 days.
Total tasks created	211	Tasks created during the SmartTask evaluation period.
Total tasks completed	185	Completed tasks during the evaluation period.
Overall task completion rate	87.7 percent	Completion rate calculated from completed tasks divided by total created tasks.
Total notifications received	194	Notifications received during testing.
Ignored or missed notification rate	16.5 percent	Rate calculated from ignored or missed notifications divided by total notifications.
Overall Likert mean	3.73 out of 5	Overall user evaluation mean score across questionnaire items.

2 DEPLOYMENT OBJECTIVES

2.1 Goals

- Deploy the completed SmartTask MVP as a working Android mobile application.
- Integrate task management, context acquisition, context matching, task prioritization, and adaptive notification delivery into a single application.
- Validate that the application can collect and process context information such as time, location, activity, calendar state, device state, and connectivity.
- Confirm that tasks are dynamically reordered based on urgency, context relevance, interruption cost, and manual priority.
- Confirm that notifications are delivered only when relevant and suitable for the current user context.
- Support basic notification actions such as Done, Snooze, and Postpone.
- Maintain privacy by storing user data locally on the device.
- Provide a stable MVP that can be demonstrated, tested, and extended in future work.

2.2 Scope

The deployment scope was defined according to the completed SmartTask MVP. The system was not deployed as a large-scale production application, but as a functional Android application for final evaluation, demonstration, and project submission.

Included in Deployment	Excluded or Future Work
Task creation, editing, deletion, completion, and manual ordering	Public Google Play Store release was not completed in the final deployment.
Voice based task input and task form prefill	Full cloud backup and cross device synchronization were not included.
Map based location selection using Google Maps and Places	Advanced behavioral learning model was not fully implemented.
Context acquisition using time, location, activity, device state, calendar state, and connectivity	The full Data Logging and Insights dashboard was not implemented as a final user facing module.

Context matching, prioritization, and adaptive notification delivery	Large scale production monitoring and analytics were not included.
Local Room database storage	A long-term control group based evaluation was not conducted.
Debug tools such as database viewer and SQL executor	Wearable device support and cross platform versions were not included.

3 PRE DEPLOYMENT ACTIVITIES

3.1 Environment Preparation

- Android Studio was installed and configured as the main development and build environment.
- Java SDK, Android SDK, Gradle, and required Android build tools were configured.
- The Android application manifest was updated with required permissions such as foreground and background location, activity recognition, calendar, notification, internet, Bluetooth, and audio recording permissions.
- Google Maps and Places configuration was added to support task location selection and location related task workflows.
- Room database classes, entities, DAOs, and migrations were reviewed for task data, context snapshots, task context matches, voice command logs, usage events, and task description insight records.
- Notification channels and notification actions were configured for Android notification delivery.
- Broadcast receivers were configured for geofence transitions, activity recognition updates, and notification actions.
- WorkManager based background processing was prepared for periodic context snapshot collection.
- Android Emulator and real Android devices were prepared for UI, sensor, permission, and notification testing.
- The final APK build process was tested using Android Studio and Gradle.

3.2 Testing Before Deployment

Before final deployment, the SmartTask system was validated through logic testing, user interface testing, integration testing, emulator testing, and real device testing. Testing focused on checking whether each module worked independently and whether the complete pipeline operated correctly from task creation to notification delivery.

- Unit testing checked task validation, scoring logic, context matching, prioritization logic, and notification decision rules.

- UI testing checked task creation, editing, deletion, completion, dashboard navigation, voice input access, and map picker usage.
- Integration testing checked the flow from context capture to context matching, prioritization, and notification dispatch.
- Permission testing checked application behavior when location, activity recognition, calendar, and notification permissions were granted or denied.
- Real device testing checked sensor-based behavior such as location detection, geofence related capture, notification actions, and background context updates.
- User evaluation testing was carried out with 8 participants over 12 to 15 active days to observe practical usage and collect feedback.

3.3 Backup Plan

- The SmartTask source code was maintained using Git and GitHub for version tracking.
- Final APK builds and important test builds were saved separately for deployment evidence.
- The local database design was documented in the final report appendix, allowing database reconstruction if required.
- Since the MVP stores data locally, no external production database backup was required.
- During testing, debug database tools were used to inspect and verify stored task records, context snapshots, and matching outputs.
- If local test data became corrupted, the application data could be cleared and testing could restart with recreated tasks.

4 DEPLOYMENT ARCHITECTURE AND RUNTIME ENVIRONMENT

4.1 Application Deployment Type

SmartTask was deployed as a native Android mobile application. The runtime environment was the Android operating system on a smartphone with access to location services, activity recognition, notifications, calendar state, connectivity information, and local storage. The deployment did not rely on a hosted backend for core task management. This made the MVP simpler, more privacy focused, and suitable for offline use.

The application launch flow starts from the Android launcher activity and redirects to the main task list screen. On boot, the application initializes required services, prepares notification channels, registers background collectors, captures an initial context snapshot, and schedules periodic context updates. This startup flow ensures that task matching and prioritization can begin soon after the application is opened.

4.2 Main Runtime Components

Component	Deployment Role
TaskListActivity	Main dashboard that displays tasks, supports add task, voice task, swipe actions, drag ordering, and completion toggles.
TaskCreateActivity	Task creation and editing screen that validates task details and saves records to the local task database.
MapPickerActivity	Google Maps based location picker that allows users to select task locations.
Voice task workflow	Captures speech, sends transcript for parsing, and prefills the task creation screen when possible.
ContextEngine	Orchestrates context collection from time, permissions, geofence, location, activity, calendar, device, connectivity, and environment collectors.

ContextSnapshotWorker	Schedules periodic context snapshots and supports continuous context awareness.
ContextMatchWorker	Starts the matching, prioritization, and notification pipeline after a context snapshot is available.
ContextMatchingRunner	Evaluates active tasks against the current context and produces relevance results.
TaskPrioritizationRunner	Ranks active tasks using urgency, context relevance, interruption cost, and manual priority.
NotificationManager and Decision Engine	Selects suitable tasks for notification delivery and prevents notification overload.
NotificationActionReceiver	Handles Done, Snooze, and Postpone actions from notifications.
Debug tools	Database viewer and SQL executor used during development and deployment testing.

4.3 Local Data Storage and Privacy

SmartTask uses local data storage through Room databases. The main task database stores task records, task context match results, voice command logs, usage events, and task description insight records. A separate context database stores context snapshots collected from the device. This separation made testing and inspection easier and supported a cleaner design.

Data Area	Stored Information
Tasks	Title, description, category, created time, updated time, due time, priority, display order, location, preferred time window, notification status, completion status, archive status, and snooze state.
Context snapshots	Time, location, accuracy, activity, calendar state, battery, charging state, power save mode, ringer mode, Do Not Disturb status, screen state, connectivity, environmental

	indicators, interruption cost, receptivity score, and context label.
Task context matches	Task ID, snapshot ID, matched time, relevance score, match reasons, trigger state, cooldown status, trigger type, source trigger, and blocked reason.
Voice command logs	Transcript, extracted JSON, and created time for voice based task creation testing.
Usage events and insights	Basic event records and task description insight records used for internal support and future extension.

The detailed database schema, including all tables and attributes used in the SmartTask system, is provided in Appendix A.

The deployment follows a privacy focused approach because the main task and context data remain on the device. Internet access is used only for selected services such as maps, places, or voice parsing when required. Core task handling and local storage do not require continuous network access.

5 DEPLOYMENT DETAILS

5.1 Date and Time

Item	Details
Deployment period	12th April 2026
Deployment type	Android MVP deployment for final evaluation and demonstration
Deployment target	Android Emulator and real Android test devices
Build output	Android APK generated from the final SmartTask project
Status	Deployment completed for final project testing and reporting

5.2 Team Members

Role	Student ID	Name
Requirement analysis, system design, Android development, context engine implementation, database design, testing, user evaluation, deployment, and reporting	IT21228612	De Silva S.J.W.

5.3 Steps and Procedures

- The final SmartTask project was opened and reviewed in Android Studio.
- Project dependencies, Gradle configuration, Java setup, and Android SDK settings were checked.
- Required Android permissions and application services were verified in the manifest.
- Google Maps and Places configuration was checked for the location picker workflow.
- Room database entities and database access logic were reviewed for task and context storage.

- Task management screens were verified, including task list, task creation, edit flow, delete flow, and completion actions.
- Voice command task creation was tested from the task list screen to the task creation form.
- Context acquisition collectors were executed through app startup, periodic worker triggers, and geofence related triggers.
- Context matching logic was tested using tasks with deadlines, priorities, locations, and notification settings.
- Task prioritization was tested by creating multiple tasks and observing display order changes after context updates.
- Notification channel creation and notification permissions were verified.
- Notification actions were tested for Done, Snooze, and Postpone.
- The final APK was generated using the Android Studio build process.
- The APK was installed on the selected Android test environment.
- End to end testing was completed from task creation to context capture, matching, prioritization, notification delivery, and notification response.
- Deployment evidence such as screenshots, system outputs, database records, and user evaluation results was prepared for documentation.
- Sample user interface screenshots of the SmartTask application are included in Appendix C for visual reference.

5.4 Tools and Technologies

Area	Tools and Technologies
Mobile application development	Android Studio, Java, Android SDK, Gradle
Local data storage	Room, SQLite
Location services	Google Maps API, Google Places API, Android Location Services, geofencing
Context acquisition	Android native APIs, Activity Recognition, Calendar access, device state APIs, connectivity APIs

Background processing	WorkManager, Broadcast Receivers
Notification delivery	Android Notification API, notification channels, notification actions
Voice task input	Android SpeechRecognizer and OpenAI based parsing for MVP support
Version control	Git, GitHub
Testing	JUnit, Espresso, Mockito, Android Emulator, real Android devices
Debugging and monitoring	Logcat, database viewer, SQL executor
UI design	Figma

6 DEPLOYMENT CHECKLIST

6.1 Pre Deployment Checklist

No.	Checklist Item	Status
1	Final Android Studio project reviewed.	Completed
2	Required permissions added and checked.	Completed
3	Google Maps and Places configuration checked.	Completed
4	Room database schema reviewed.	Completed
5	Task management module completed.	Completed
6	Voice task input workflow tested.	Completed
7	Context acquisition engine configured.	Completed
8	Geofence receiver and activity receiver configured.	Completed
9	Notification channel and action receivers configured.	Completed
10	Emulator and real device testing environment prepared.	Completed

6.2 Deployment Checklist

No.	Checklist Item	Status
1	Final APK generated.	Completed
2	APK installed on Android test environment.	Completed
3	Application launch verified.	Completed
4	Task list dashboard verified.	Completed
5	Task creation, editing, deletion, and completion verified.	Completed
6	Voice task creation verified.	Completed
7	Map based location picker verified.	Completed
8	Context snapshot capture verified.	Completed
9	Task matching and prioritization verified.	Completed
10	Adaptive notification delivery verified.	Completed
11	Notification actions verified.	Completed

6.3 Post Deployment Checklist

No.	Checklist Item	Status
1	Application health check completed.	Completed
2	Logcat checked for runtime errors.	Completed
3	Task database records verified.	Completed
4	Context snapshot records verified.	Completed
5	Task context match records verified.	Completed
6	User permissions tested.	Completed
7	Performance and responsiveness checked.	Completed
8	User evaluation data reviewed.	Completed
9	Final screenshots and evidence captured.	Completed
10	Deployment report prepared.	Completed

7 ISSUES AND RESOLUTIONS

7.1 Encountered Issues

Issue	Description
Location permission handling	Location based task matching and geofence triggers depend on foreground and background location permissions. If permissions are denied, these features work with reduced capability.
Geofence trigger delay	Android may delay geofence events because of background execution and battery optimization rules.
Activity recognition uncertainty	Activity recognition can return low confidence or unknown states depending on user movement and device conditions.
Notification permission requirement	Newer Android versions require explicit notification permission before reminders can be displayed.
Battery usage during context collection	Frequent context checks can increase resource usage if location and sensor collection are not controlled.
Voice command parsing limitations	Spoken input can be incomplete, unclear, or affected by background noise, which may produce incomplete task fields.
Task prioritization transparency	Some users may not immediately understand why a task is ranked above another task.
Limited Data Logging and Insights	The original design included a user facing analytics module, but the final MVP focused on core context aware task management.

7.2 Resolutions

Issue Area	Resolution Applied
Permission handling	The app checks permissions and continues with limited functionality when permission access is unavailable.
Geofence delay	Periodic context snapshots were used together with geofence based events to reduce dependency on immediate geofence triggers.
Activity uncertainty	Activity data was used only when available and useful. Other context signals and default priority logic supported fallback behaviour.
Notification permission	Notification permission was checked before dispatching reminders to avoid runtime issues.
Battery and resources	Background collection was controlled through scheduled snapshot workers and event-based triggers instead of continuous sensor usage.
Voice parsing	Fallback behaviour allowed the user to manually edit task details if parsing did not produce complete results.
Prioritization transparency	The system stored match reasons internally, and future UI improvements can expose labels such as due soon, near location, or high priority.
Analytics limitation	The limitation was documented clearly, and the module was identified as future work.

7.3 Impact Analysis

The encountered issues did not prevent the SmartTask MVP from operating successfully. Most issues were related to normal Android platform behaviour, including permission control, background processing limits, sensor accuracy, and notification policies. These issues were managed using fallback logic, scheduled context collection, permission checks, and conservative notification filtering.

The impact on users was reduced because the application continued to support core task management even when some context permissions were unavailable. The system remained usable as a normal task manager while providing additional context aware features when required permissions and services were available. The deployment therefore remained stable and suitable for evaluation as a working MVP.

The main unresolved impact was related to long term intelligence and analytics. Since advanced behavioural learning and a full Data Logging and Insights dashboard were not completed, the deployed version could not provide detailed productivity trends or long term adaptation. This limitation was acceptable within the MVP scope and is listed as future work.

8 PERFORMANCE AND MONITORING

8.1 Performance Metrics

Metric	Result	Interpretation
Task completion rate	87.7 percent	185 tasks were completed out of 211 created tasks during the evaluation period.
Mean participant completion rate	88.6 percent	Average completion performance across participants was positive.
Ignored or missed notification rate	16.5 percent	Most notifications were noticed or acted on during the evaluation.
Overall Likert mean	3.73 out of 5	Users generally accepted the application.
Usability and navigation mean	3.88 out of 5	Users found the application understandable and usable.
Context awareness and prioritization mean	3.71 out of 5	Users found the core context aware idea useful, but requested clearer explanation of ranking.
Notification usefulness and timing mean	3.84 out of 5	Users found reminders useful, especially when linked to time and location.
Productivity, effort, satisfaction, and adoption mean	3.50 out of 5	Users saw value, but stronger long term proof requires larger evaluation.

Notification limit per processing cycle	Maximum 3 notifications	The system limits notifications to reduce overload.
Offline support	Supported for core task functions	Local storage allows task management without continuous internet access.

8.2 Monitoring Results

Monitoring during deployment and testing was performed using development and debug tools rather than a production monitoring server. Since the SmartTask MVP was deployed as a local Android application, runtime monitoring focused on application logs, database records, sensor behaviour, notification behaviour, and user feedback.

Component or Area	Monitoring Tool or Method	Observation
Application runtime	Android Studio Logcat	Logs were checked for crashes and runtime exceptions during test execution.
Task database	Built in database viewer and SQL executor	Task records, completion state, snooze state, and display order were inspected.
Context snapshots	Database viewer and SQL executor	Collected context values were reviewed during app startup, periodic capture, and event based triggers.
Task context matches	Database viewer and internal match records	Relevance scores, trigger states, blocked reasons, and match reasons were checked.
Notifications	Manual device testing	Notification display and actions such as Done,

		Snooze, and Postpone were verified.
Permissions	Android permission settings and runtime checks	Location, activity, calendar, notification, and microphone permission behaviour was verified.
User experience	Questionnaire and open ended feedback. All participants provided informed consent before taking part in the evaluation, and the consent form used is included in Appendix D. The questionnaire used for collecting user feedback was distributed through a QR code, which is provided in Appendix E.	Users reported that context aware reminders were useful, but timing and transparency can be improved.

8.3 User Feedback

The user evaluation was conducted with 8 valid participants, including 4 students and 4 working professionals. Participants used the app for 12 to 15 days, created 211 tasks, completed 185 tasks, and received 194 notifications. The evaluation results show that SmartTask was accepted as a useful MVP for daily task management. The complete dataset, including raw responses, calculated metrics, and detailed analysis, is available in the Excel workbook provided in Appendix F.

- Users liked location based and context aware reminders because tasks became more relevant at the right place or time.
- Several users mentioned that the app reduced the need to repeatedly check the task list manually.

- Users found navigation and basic task handling easy to understand.
- The notification system was useful, but some users experienced delayed reminders or wanted better control over notification timing.
- Some users wanted clearer reasons for task ordering, such as due soon, near location, or high priority.
- The short testing period was not enough to strongly prove long term productivity improvement, but it was enough to support the MVP concept.

9 TEAM DETAILS

The SmartTask project was completed as an individual project. All major activities were handled by the student, including requirement analysis, system design, application development, testing, evaluation, deployment, and report preparation.

ID Number	Name	Email	Main Contribution
IT21228612	De Silva S.J.W.	it21228612@my.sliit.lk	Complete SmartTask Android application development, deployment, testing, evaluation, and documentation.

APPENDIX

Appendix - A: Database Design

Table Name : tasks	
property name	**data type**
id	long
title	String
description	String (nullable)
category	String
createdAt	long
updatedAt	long
dueAt	Long (nullable)
priority	int
displayOrder	long
locationLat	Double (nullable)
locationLng	Double (nullable)
locationRadiusM	Float (nullable)
locationLabel	String (nullable)
estimatedDurationMin	Integer (nullable)
preferredStartTime	Long (nullable)
preferredEndTime	Long (nullable)
notificationsEnabled	boolean
completed	boolean
completedAt	Long (nullable)
archived	boolean
snoozeUntil	Long (nullable)

Table Name : context_snapshots	
property name	**data type**
id	long
timestamp	long
timezoneId	String
dayOfWeek	int
minuteOfDay	int
lat	Double (nullable)
lng	Double (nullable)
accuracyM	Float (nullable)
speedMps	Float (nullable)
bearingDeg	Float (nullable)
isGeofenceHit	boolean
geofenceId	String (nullable)
placeLabel	String (nullable)
locationSource	String

activityType	String (nullable)
activityConfidence	int
isMoving	boolean
stepsSinceLast	Integer (nullable)
isInMeeting	boolean
currentEventId	String (nullable)
currentEventTitle	String (nullable)
eventBusyStatus	String (nullable)
minutesToNextEvent	Integer (nullable)
minutesLeftInEvent	Integer (nullable)
batteryPct	int
isCharging	boolean
powerSaveMode	boolean
ringerMode	String (nullable)
doNotDisturbOn	boolean
screenOn	boolean
deviceUnlocked	boolean
appInForeground	boolean
connectivityType	String (nullable)
isInternetAvailable	boolean
isRoaming	Boolean (nullable)
headphonesConnected	Boolean (nullable)
bluetoothConnected	Boolean (nullable)
wifiSsidHash	String (nullable)
noiseLevelDb	Float (nullable)
interruptionCostScore	float
receptivityScore	float
contextLabel	String (nullable)
permissionState	String
dataQualityFlags	int
anonymized	boolean
sourceTrigger	String

Table Name : task_context_matches	
property name	**data type**
id	long
taskId	long
snapshotId	long
matchedAt	long
relevanceScore	float
matchReasons	String
shouldTriggerNow	boolean
cooldownUntil	Long (nullable)
triggerType	String

contextSnapshotTrigger	String (nullable)
blockedBy	String (nullable)

Table Name : usage_stat_events	
property name	**data type**
id	long
eventType	String
taskId	Long (nullable)
notificationId	Integer (nullable)
createdAt	long

Table Name : task_description_insights	
property name	**data type**
id	long
taskId	long
descriptionHash	String
riskIfDelayed	float
requiresDeepFocus	boolean
suggestedDurationMin	Integer (nullable)
complexityLevel	int
energyDemand	int
relationshipValue	int
rawJson	String (nullable)
model	String (nullable)
updatedAt	long

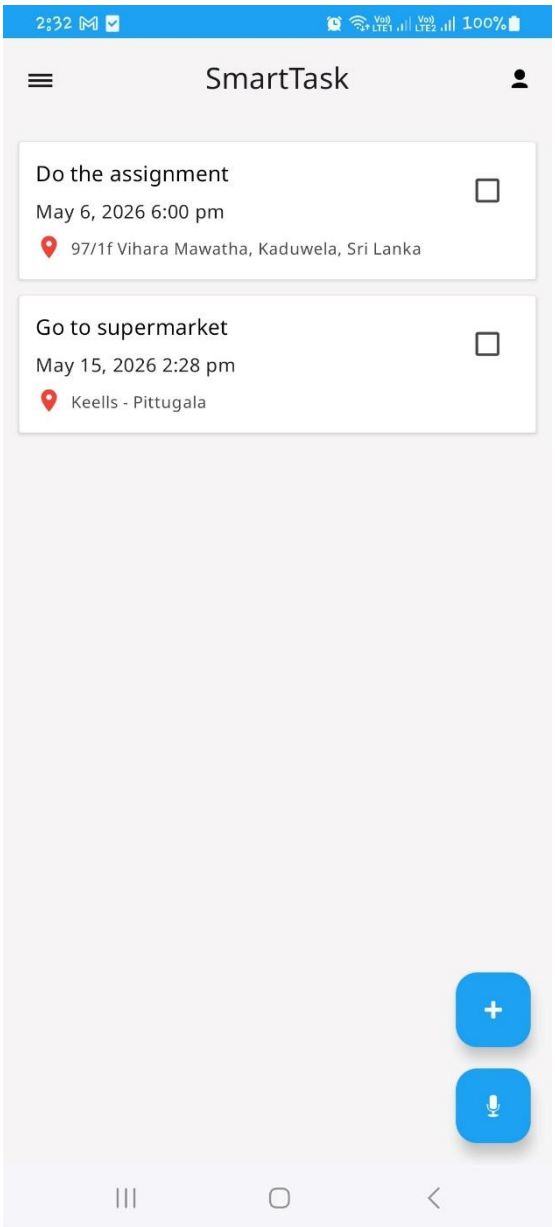
Table Name : voice_command_logs	
property name	**data type**
id	long
transcript	String
extractedJson	String
createdAt	long

Appendix - B: Codebase GitHub Repository Link

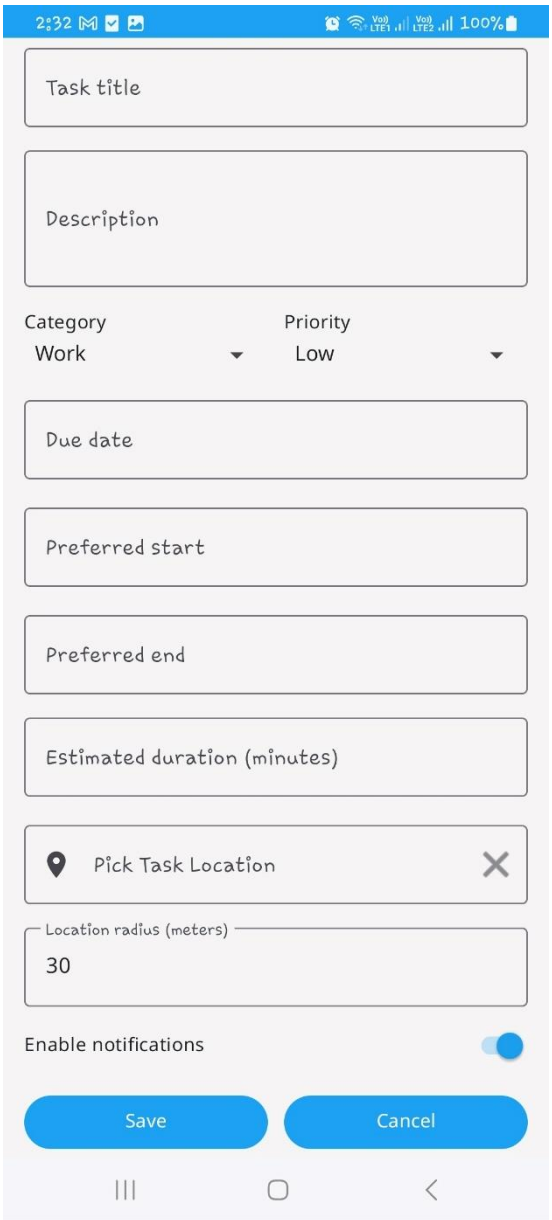
This appendix provides the GitHub repository link for the SmartTask application source code. The repository contains the Android project files, and related implementation files.

Repository Link: https://github.com/IT21228612/SmartTask

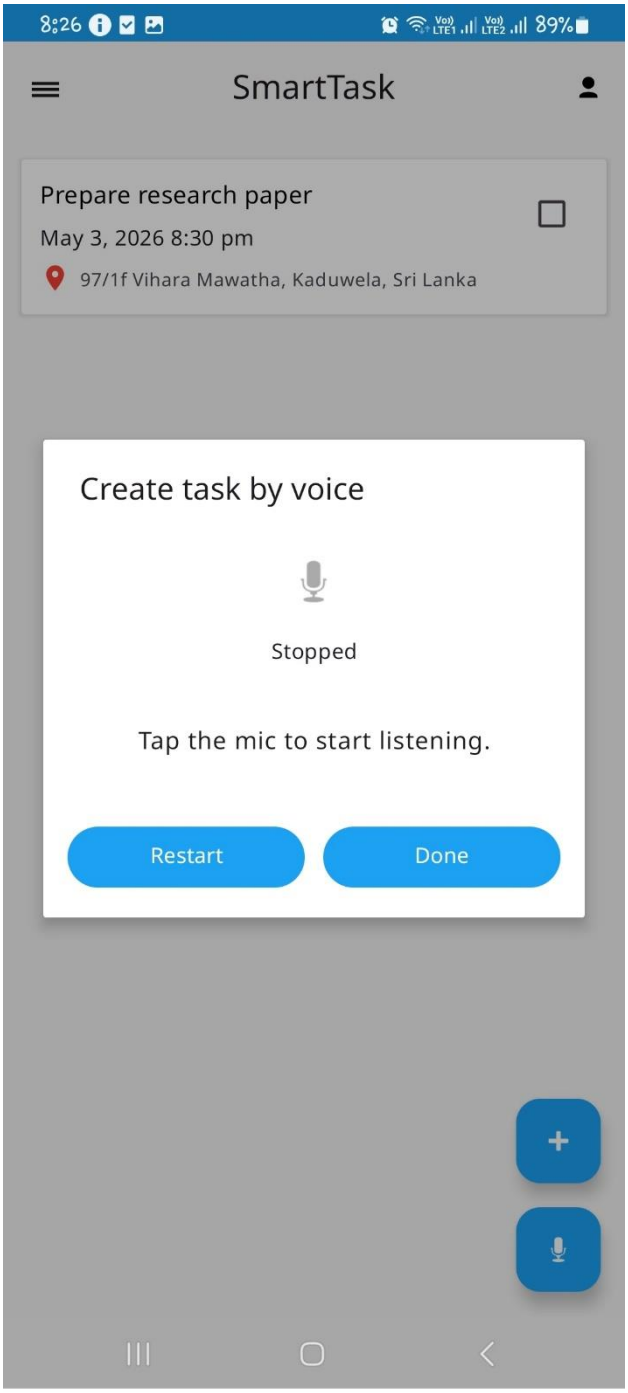
Appendix - C: App UI Screenshots



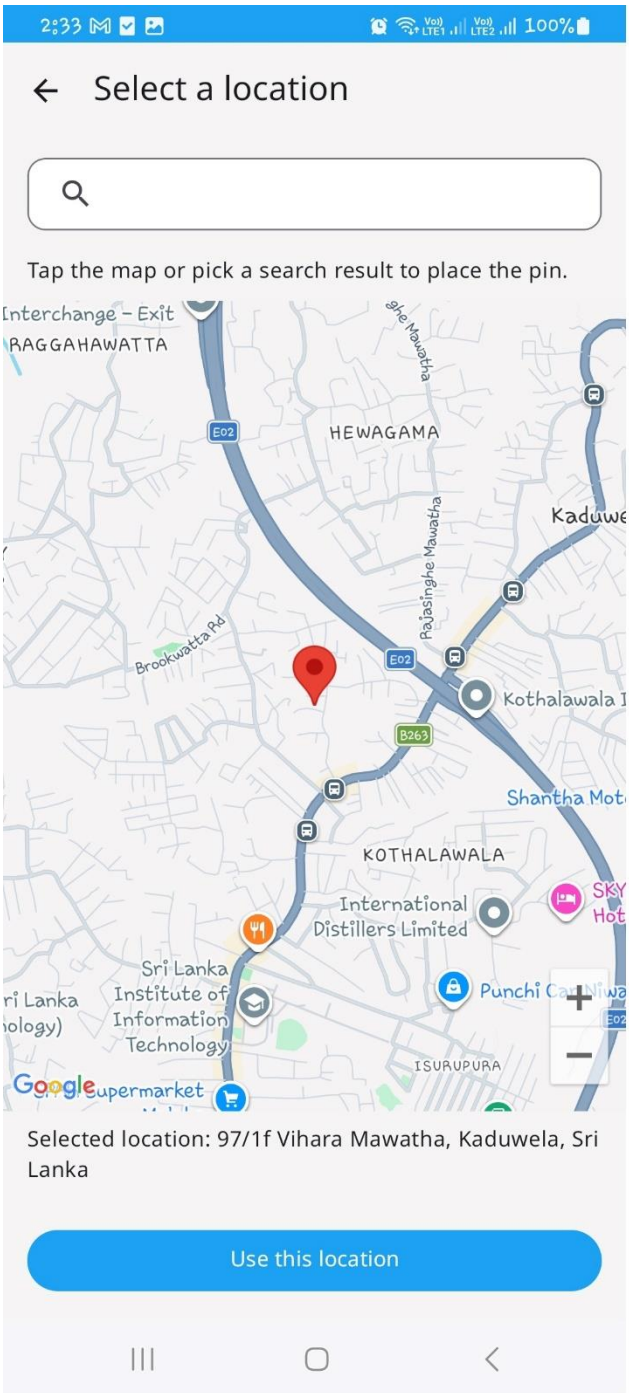
main UI - task list



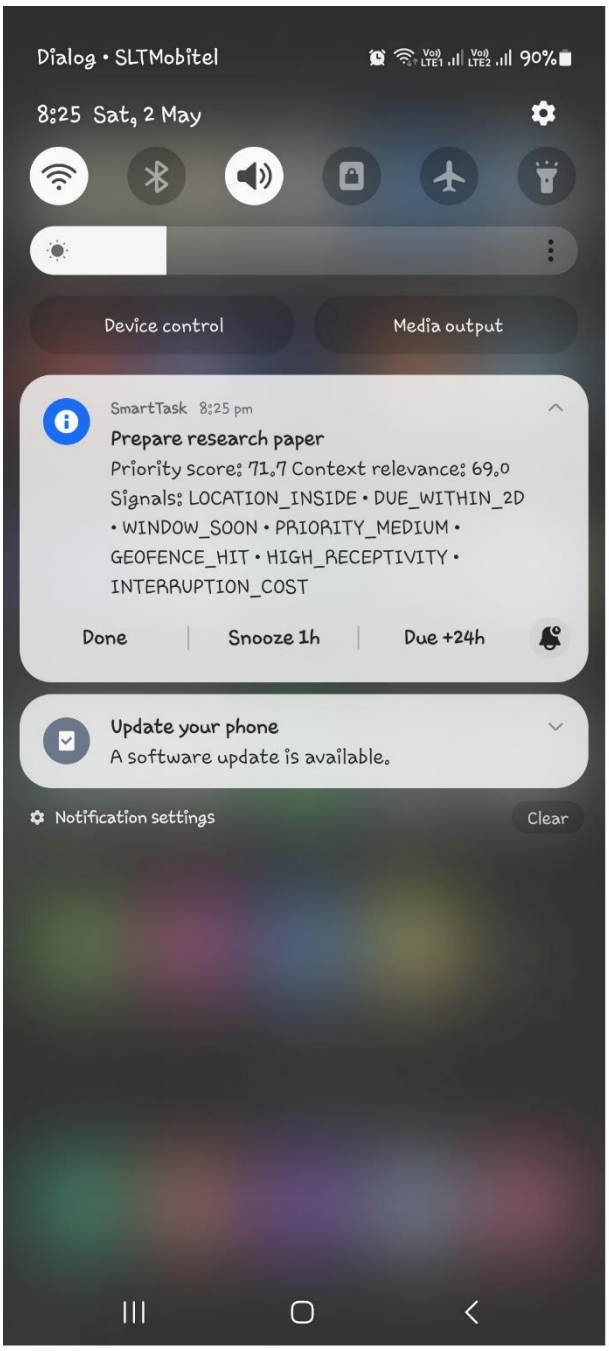
task creation UI



Voice Command Task Creation



Location pick UI



App Notification

Appendix - D: User Evaluation Participation Consent Form

User Participation Consent Form

SmartTask Application Evaluation Study

This document confirms voluntary participation in the evaluation of the **SmartTask: Context-Aware Mobile Task Management Application.**

The purpose of this study is to assess the usability, effectiveness, and user satisfaction of the SmartTask application in real-world usage conditions.

Survey Access

Participants completed the evaluation questionnaire via the link/QR code below:

Scan to access the questionnaire:

Scan me!

Participant Declaration

I confirm that:

- I have voluntarily participated in this study
- I have used the SmartTask application during the testing period
- I have interacted with the application in my daily routine (e.g., creating tasks, responding to notifications)
- I have provided my honest feedback through the online questionnaire
- I understand that my responses will be used only for academic purposes
- I understand that my personal identity will not be disclosed in any report or publication

Scanned with CamScanner

Participant Record

Participant ID	Occupation / Category	Date	Signature
P1	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	29/04/2026	
P2	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	29/04/2026	
P3	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	29/04/2026	
P4	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	30/04/2026	
P5	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	30/04/2026	
P6	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	2026/04/30	
P7	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	2026/04/30	
P8	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	2026-04-30	

Researcher Declaration

The data collected from participants is used strictly for academic evaluation of the SmartTask system. All responses are handled with confidentiality, and no personally identifiable information is included in the research project documents.

Researcher:
De Silva S.J.W.
BSc (Hons) Information Technology (Software Engineering)
Sri Lanka Institute of Information Technology

2026/04/29
Signature and Date

Scanned with CamScanner

Appendix - E: User Evaluation Questionnaire (QR Code)

Appendix - F: User Evaluation Summary – Excel Workbook (Link)

This appendix provides the full Excel workbook used to summarize the SmartTask user evaluation results. The workbook includes the raw questionnaire responses, overall dashboard, usage metrics, Likert scale analysis, open ended feedback summary, and final interpretation notes.

The evaluation was based on 8 valid participant responses collected during the SmartTask testing period. The workbook summarizes key results such as 109 total active testing days, 211 tasks created, 185 tasks completed, an overall task completion rate of 87.7 percent, 194 notifications received, and an ignored or missed notification rate of 16.5 percent.

It also includes the Likert scale category results for usability and navigation, context awareness and prioritization, notification usefulness and timing, and productivity, effort, satisfaction and adoption. In addition, the workbook summarizes open ended feedback themes, including liked features, difficulties faced by users, and suggested improvements for future development.

Excel Workbook Link: https://mysliit-my.sharepoint.com/:x:/g/personal/it21228612_my_sliit_lk/IQBDQKBH8iiGQIdJewT00a_XAUmAtutbaxsw06lOyb5W87k?e=6GdLrv